

Parallelizing the Variational Quantum Eigensolver: From JIT Compilation to Multi-GPU Scaling

R. Malarchick¹ and A. Steed¹

¹Department of Physical Sciences, Embry-Riddle Aeronautical University,
Daytona Beach, FL 32114, USA

rylan1012@gmail.com, steeda1@my.erau.edu

January 2026; revised October 2026

Abstract

The Variational Quantum Eigensolver (VQE) is a hybrid quantum-classical algorithm for ground-state energies of molecular systems. We compute the potential energy surface of the hydrogen molecule (H_2) at 100 bond lengths with PennyLane on the ERAU Vega cluster (AMD EPYC 9654 CPUs, 4 NVIDIA H100 GPUs) and compare a serial baseline, an Optax and Catalyst JIT version, a `lightning.gpu` version, and an MPI version. Early stopping, a larger learning rate, and warm starts reduce the iteration count from 30,000 to 7,033 and the runtime from 593.95 s to 143.80 s, while the time per iteration stays between 0.0198 s and 0.0204 s. At 4 qubits, `lightning.gpu` lowers the time per iteration to 0.0145 s. MPI runs with 2 to 32 ranks finish in 8.45 s to 5.04 s; the serial and MPI jobs ran under different core allocations, so we do not report their ratio as a parallel speedup. In a 4 to 26 qubit study with different CPU and GPU software stacks, the CPU/GPU time ratio ranges from 3.5 to 81.5. One H100 runs up to 29 qubits.

1 Introduction

1.1 Background

Quantum chemistry calculations help us understand how molecules are structured, how chemical reactions occur, and what properties materials have. A central problem in computational chemistry is finding the ground state energy (lowest energy configuration) and wavefunction (quantum state description) of a molecule. However, exact quantum mechanical calculations become exponentially harder as molecules get larger, making traditional computer methods impractical for large molecules.

The Variational Quantum Eigensolver (VQE) is a promising quantum algorithm that combines both quantum and classical computing [1]. Unlike purely quantum algorithms that need perfect quantum computers, VQE works on today’s noisy quantum computers. The algorithm uses a quantum circuit (called an ansatz) with adjustable parameters to create trial wavefunctions on a quantum processor, while a classical computer adjusts these parameters to find the lowest energy.

We focus on the hydrogen molecule (H_2), the simplest neutral molecule, which serves as a benchmark system for quantum chemistry methods. Despite its simplicity, H_2 exhibits key features of chemical bonding including equilibrium bond length, dissociation energy, and potential energy surface structure.

1.2 Issues and Questions to be Addressed

This work addresses two primary questions:

1. **Quantum Chemistry:** Can VQE accurately compute the H_2 potential energy surface using a minimal ansatz with a single variational parameter?
2. **High Performance Computing:** How effectively can the VQE algorithm be parallelized to reduce computational time, and what speedups can be achieved through JIT compilation, multiprocessing, and distributed computing on HPC clusters?

The serial implementation provides the baseline for the other implementations.

1.3 Related Work

Peruzzo et al. [1] first demonstrated VQE experimentally on a photonic quantum processor. Since then, ansatz design has been a major focus: the Unitary Coupled Cluster (UCC) ansatz [2] is now standard for molecular simulations, while hardware-efficient ansatzes [3] reduce circuit depth on noisy devices.

On the classical simulation side, several tools have emerged for accelerating variational quantum algorithms. PennyLane [4] provides automatic differentiation for quantum circuits, and its Lightning backend [5] offers optimized state-vector simulation. NVIDIA’s cuQuantum [6] targets multi-GPU simulation, and JAX [7] enables JIT compilation for numerical code.

MPI parallelization of quantum chemistry is well-established [8]. We apply similar ideas to VQE, benchmarking JIT compilation, GPU acceleration, and MPI distribution on modern hardware. Our contribution is a comparison of these techniques on a single HPC node with multiple GPUs, with each runtime reported together with its optimizer and stopping settings.

2 Problem Description

2.1 The Molecular Hamiltonian Problem

The goal is to compute the ground state energy E_0 of the H_2 molecule as a function of internuclear distance d . The electronic Hamiltonian in the Born-Oppenheimer approximation is:

$$H = -\frac{1}{2} \sum_{i=1}^2 \nabla_i^2 - \sum_{i=1}^2 \left(\frac{1}{|\mathbf{r}_i - \mathbf{R}_A|} + \frac{1}{|\mathbf{r}_i - \mathbf{R}_B|} \right) + \frac{1}{|\mathbf{r}_1 - \mathbf{r}_2|} + \frac{1}{d} \quad (1)$$

where $\mathbf{r}_i$ are electron positions, $\mathbf{R}_A$ and $\mathbf{R}_B$ are nuclear positions separated by distance d , and atomic units are used.

This continuous-space Hamiltonian must be converted to a finite basis set (we use STO-3G, a minimal basis set) and then transformed into qubit operators that quantum computers can work with using a method called the Jordan-Wigner transformation.

2.2 Computational Task

The specific computational problem is:

- **Input:** Set of bond lengths $\{d_1, \dots, d_{100}\}$ uniformly spaced from 0.35 to 3.0 Å
- **Output:** Ground state energies $\{E_1, \dots, E_{100}\}$ at each bond length

- **Constraint:** Each energy must converge (the baseline runs 300 VQE iterations per bond length)
- **Objective:** Minimize total wall-clock time while maintaining accuracy

The key computational challenge is that each bond length requires:

- Hartree-Fock calculation to generate molecular Hamiltonian
- Up to 300 optimization steps, each with a cost and a gradient evaluation
- Parameter updates via Adam optimizer

On the cluster, the baseline runs 30,000 optimization steps in total (Section 5.2).

3 Model Formulation

3.1 The Variational Principle

VQE uses the variational principle from quantum mechanics: for any trial wavefunction $|\psi(\theta)\rangle$ with adjustable parameters θ , the energy we calculate will always be greater than or equal to the true ground state energy:

$$E(\theta) = \langle \psi(\theta) | H | \psi(\theta) \rangle \geq E_0 \quad (2)$$

where E_0 is the true ground state energy and H is the molecular Hamiltonian. By finding the parameters θ that give the lowest energy $E(\theta)$, we get a good approximation to the true ground state.

3.2 Molecular Hamiltonian in Second Quantization

For the H_2 molecule, the electronic Hamiltonian in second quantization is:

$$H = \sum_{i,j} h_{ij} a_i^\dagger a_j + \frac{1}{2} \sum_{i,j,k,\ell} h_{ijkl} a_i^\dagger a_j^\dagger a_k a_\ell \quad (3)$$

where:

- h_{ij} are one-electron integrals (kinetic energy and nuclear attraction)
- h_{ijkl} are two-electron integrals (electron-electron repulsion)
- $a_i^\dagger, a_i$ are fermionic creation and annihilation operators

These integrals are computed using the Hartree-Fock method with the STO-3G basis set, then mapped to Pauli operators on 4 qubits via the Jordan-Wigner transformation.

3.3 Quantum Circuit Ansatz

The trial wavefunction is prepared using a parameterized quantum circuit:

$$|\psi(\theta)\rangle = U(\theta)|\text{HF}\rangle \quad (4)$$

where:

- $|\text{HF}\rangle = |1100\rangle$ is the Hartree-Fock reference state (both electrons in lowest spatial orbital with opposite spins)
- $U(\theta)$ is a unitary operator implemented as a double excitation gate

The double excitation gate is:

$$U(\theta) = \exp\left(-i\frac{\theta}{2}(a_0^\dagger a_1^\dagger a_2 a_3 - a_3^\dagger a_2^\dagger a_1 a_0)\right) \quad (5)$$

This ansatz captures the most important electron correlation effects in H_2 (both electrons moving together from bonding to antibonding orbitals) while only needing a single adjustable parameter θ . From a computational perspective, this gate is a parameterized unitary matrix applied to the 2^N -dimensional state vector. The optimization problem reduces to finding the parameter θ that minimizes a matrix expectation value, which maps naturally to gradient-based optimization.

3.4 Optimization Problem

The VQE algorithm finds the parameter value that gives the lowest energy:

$$\theta^* = \arg \min_{\theta} E(\theta) = \arg \min_{\theta} \langle \psi(\theta) | H | \psi(\theta) \rangle \quad (6)$$

We use the Adam optimizer with:

- Learning rate: $\alpha = 0.01$
- Iterations per bond configuration: $N_{\text{iter}} = 300$ (baseline; Section 5.2 lists the settings of the other implementations)
- Initial parameter: $\theta_0 = 0$ (starts at Hartree-Fock state)

4 Methods

4.1 Problem Structure and Parallelization Opportunities

The computational task consists of computing the potential energy surface by evaluating $E(\theta^*)$ for $N_b = 100$ bond lengths in the range $[0.35, 3.0]$ Å. For each bond length d_i :

1. Generate molecular Hamiltonian $H(d_i)$ using Hartree-Fock
2. Initialize variational parameters $\theta_0 = 0$
3. Optimize: $\theta_i^* = \text{Adam}(E(\theta), \theta_0, N_{\text{iter}} = 300)$
4. Store ground state energy $E_i = E(\theta_i^*)$

These calculations are **embarrassingly parallel**: each bond length calculation is independent, requiring no data from other calculations:

$$E_i = f(d_i) \quad \text{for } i = 1, \dots, 100 \quad (7)$$

where f is the VQE optimization procedure.

4.2 Serial Algorithm Implementation

The baseline serial implementation follows Algorithm 1.

Algorithm 1 Serial VQE for H₂ Potential Energy Surface

```

1: Input: Bond lengths  $\{d_1, \dots, d_{100}\}$ 
2: Output: Energies  $\{E_1, \dots, E_{100}\}$ 
3:
4: Initialize quantum device: lightning.qubit with 4 qubits
5: Define ansatz with Hartree-Fock initialization
6:
7: for  $i = 1$  to 100 do
8:   Generate  $H(d_i)$  using Hartree-Fock (STO-3G basis)
9:    $\theta \leftarrow 0$ 
10:  Initialize Adam optimizer with  $\alpha = 0.01$ 
11:  for  $j = 1$  to 300 do
12:     $E \leftarrow \langle \psi(\theta) | H(d_i) | \psi(\theta) \rangle$  ▷ Quantum circuit evaluation
13:     $\nabla_\theta E \leftarrow$  compute gradient (PennyLane default method for lightning.qubit)
14:     $\theta \leftarrow \text{Adam\_step}(\theta, \nabla_\theta E)$ 
15:  end for
16:   $E_i \leftarrow E(\theta)$  ▷ Store converged energy
17: end for
18: return  $\{E_1, \dots, E_{100}\}$ 

```

Implementation Details:

- **Software Versions:** Python 3.12, PennyLane 0.43.1, PennyLane-Catalyst 0.13.0, JAX 0.6.2, Optax 0.2.6, CUDA 11.8, OpenMPI 4.x
- **Basis Set:** STO-3G minimal basis (4 spin-orbitals $\rightarrow$ 4 qubits)
- **Hamiltonian Method:** DHF (built-in Hartree-Fock solver)
- **Device:** PennyLane Lightning simulator (CPU: `lightning.qubit`, GPU: `lightning.gpu`)
- **Gradient Method:** Automatic differentiation via PennyLane/Catalyst

4.3 Computational Complexity

Each quantum circuit evaluation requires $O(4^n)$ operations for an n -qubit system using classical simulation. For our 4-qubit system:

- State vector dimension: $2^4 = 16$ complex amplitudes

- Operations per circuit: $O(16^2) = O(256)$ for state preparation and measurement
- Gradient evaluations: one per optimization step
- Optimization steps per bond length: up to 300
- Total optimization steps: up to 30,000

4.4 Proposed Parallelization Approaches

We propose a three-phase optimization strategy:

4.4.1 Phase 1: JIT Compilation with JAX

Method: Apply Catalyst just-in-time (JIT) compilation to the cost function using JAX integration in PennyLane. Optax, a JAX-compatible optimizer library, is used in place of PennyLane’s built-in Adam optimizer.

```

1 @jax.jit
2 @qml.qnode(dev, interface="jax")
3 def cost_fn(params):
4     ansatz(params)
5     return qml.expval(H)
6
7 # Optimization step
8 grads = jax.grad(cost_fn)(params)
9 updates, opt_state = optimizer.update(grads, opt_state)
10 params = optax.apply_updates(params, updates)

```

Listing 1: JIT-compiled VQE cost function with Optax optimizer.

Expected Speedup: 2–5× from:

- Pre-compiling the circuit and optimization for faster execution
- Combining operations to reduce memory access time
- Computing gradients more efficiently using vector operations

4.4.2 Phase 2: Distributed-Memory Parallelism

Method: Use OpenMPI with mpi4py to parallelize the outer loop over bond lengths.

```

1 from mpi4py import MPI
2 comm = MPI.COMM_WORLD
3 rank, size = comm.Get_rank(), comm.Get_size()
4
5 if rank == 0:
6     bond_lengths = np.linspace(0.35, 3.0, 100)
7     chunks = np.array_split(bond_lengths, size)
8 else:
9     chunks = None
10
11 my_chunk = comm.scatter(chunks, root=0)
12 my_results = [run_vqe(d) for d in my_chunk]

```

```
13 | all_results = comm.gather(my_results, root=0)
```

Listing 2: MPI scatter-gather pattern for embarrassingly parallel VQE.

Each process runs JIT-compiled VQE independently on its assigned subset of bond lengths, with results gathered to the root process for aggregation.

Expected Speedup: $0.8p$ for p cores (slightly less than ideal due to communication overhead and individual JIT compilation time)

4.4.3 Phase 3: Why MPI Over Ray

We initially considered the Ray distributed computing framework for multi-node parallelization due to its high-level task-based API. However, during development we encountered persistent dependency conflicts between Ray and the PennyLane/Catalyst/JAX ecosystem. The Ray pip package repeatedly failed to install alongside PennyLane-Catalyst due to incompatible transitive dependencies, and attempts to resolve version constraints proved time-consuming without success.

Given these practical constraints, we selected MPI (via `mpi4py`) as our distributed-memory solution. MPI offered several advantages for our use case:

- **Mature HPC integration:** Native support on the ERAU Vega cluster with optimized OpenMPI
- **Minimal overhead:** Direct scatter-gather communication patterns with no daemon processes
- **Proven compatibility:** No conflicts with PennyLane, Catalyst, or JAX packages
- **Static workload fit:** Our embarrassingly parallel workload (fixed bond lengths) does not require Ray’s dynamic task scheduling

The MPI implementation uses the same JIT-compiled VQE runner as Phase 1, with each MPI rank independently compiling and executing its assigned subset of bond lengths. Results are gathered to the root process for aggregation and plotting.

Expected Speedup: Near-linear scaling for $p \leq N_b$ processes, where $N_b = 100$ is the number of bond lengths

4.5 Performance Prediction Model

Using Amdahl’s law to predict strong scaling with p processors:

$$S_p = \frac{1}{f_s + \frac{f_p}{p}} \quad (8)$$

where:

- $f_s \approx 0.05$ is the serial fraction (initialization, I/O, plotting)
- $f_p \approx 0.95$ is the parallel fraction (VQE optimizations)

Predicted speedups are shown in Table 1.

Processors	Ideal Speedup	Predicted Speedup
4	4.0×	3.48×
8	8.0×	6.15×
16	16.0×	10.39×
40	40.0×	18.87×

Table 1: Predicted parallel speedup using Amdahl’s law with $f_s = 0.05$.

5 Solution

5.1 Serial Implementation Results

The serial VQE implementation computes the H_2 potential energy surface across 100 bond lengths. Table 2 lists the metrics of a preliminary run with 8,000 circuit evaluations, a smaller configuration than the 30,000-evaluation cluster runs in Section 5.2. The two sets of numbers are not comparable.

Metric	Value
Total Runtime	50.64 seconds
Time per Bond Length	1.27 seconds
Time per VQE Iteration	6.3 ms
Circuit Evaluations/sec	157.98
Total Circuit Evaluations	8,000

Table 2: Preliminary serial run (8,000 circuit evaluations).

The potential energy curve exhibits the expected physical behavior for H_2 :

- Bonding region at small bond lengths ($d < 0.74 \text{ \AA}$)
- Equilibrium bond length near $d_{\text{eq}} \approx 0.74 \text{ \AA}$
- Dissociation to separated atoms at large distances ($d > 2.5 \text{ \AA}$)

Figure 1 shows the computed potential energy surface.

5.2 Parallel Implementation Results

We ran all implementations on the ERAU Vega HPC cluster (AMD EPYC 9654 CPUs, 4 NVIDIA H100 GPUs). Every H_2 run scans the same 100 bond lengths with at most 300 VQE iterations per bond length. The baseline always runs 300 iterations. The Optax versions stop earlier when the energy converges (Implementation 2).

5.2.1 Hardware Platform

Compute Node (gpu01):

- CPU: 2× AMD EPYC 9654 (96 cores each, 192 cores per node)
- Memory: 1.5 TB shared memory
- GPU: 4× NVIDIA H100 PCIe (81 GB each, 320 GB total)
- Interconnect: High-performance cluster interconnect

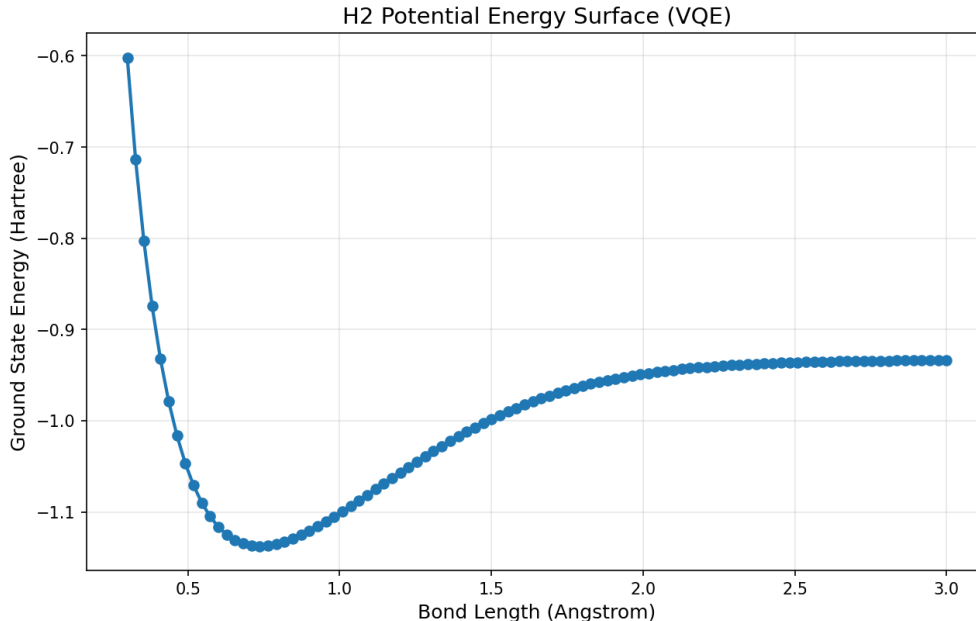


Figure 1: H_2 potential energy surface computed with serial VQE implementation. The curve shows the characteristic bonding minimum near 0.74 Å and dissociation behavior at large bond lengths.

5.2.2 Implementation 1: Serial Baseline with PennyLane

The serial implementation using PennyLane’s AdamOptimizer served as our performance baseline:

- **Runtime:** 593.95 seconds (9.90 minutes)
- **Time per bond length:** 5.94 seconds
- **Framework:** PennyLane 0.43.1 with Lightning CPU backend

5.2.3 Implementation 2: Serial Optax and Catalyst JIT (CPU)

The second implementation (`vqe_serial_optax.py`) compiles the full optimization loop with Catalyst (`@qjit`, `catalyst.grad`, `catalyst.while_loop`) and uses the Optax Adam optimizer. It also differs from the baseline in three settings. The learning rate is 0.05 instead of 0.01. The loop stops when the energy changes by less than 10^{-8} Ha between steps, with at most 300 steps. Each bond length starts from the optimized parameters of the previous bond length instead of from zero.

- **Runtime:** 143.80 seconds (baseline: 593.95 seconds)
- **Iterations:** 7,033 in total (baseline: 30,000)
- **Time per iteration:** 0.0204 seconds (baseline: 0.0198 seconds)
- **Device:** `lightning.qubit` (CPU backend)

The runtime falls by a factor of 4.13, and the iteration count falls by a factor of 4.3. The time per iteration is the runtime divided by the iteration count, and it includes the Hartree-Fock Hamiltonian construction for each bond length. Because this per-bond cost is spread over about 70

iterations per bond length in the JIT run and over 300 in the baseline, the time per iteration does not isolate the cost of the compiled circuit. The measurements show that the runtime reduction tracks the iteration count. We did not run the four changed settings one at a time, so we cannot divide the reduction among them.

5.2.4 Implementation 3: GPU Acceleration

The third implementation (`vqe_gpu.py`) uses PennyLane’s `lightning.gpu` device with the Optax Adam optimizer, a learning rate of 0.05, and the same stopping rule as Implementation 2, without Catalyst. Every bond length starts from zero.

- **Runtime:** 164.91 seconds
- **Iterations:** 11,345 in total
- **Time per iteration:** 0.0145 seconds
- **Device:** `lightning.gpu` (NVIDIA H100)

The GPU run has the lowest time per iteration of the three implementations (0.0145 s, against 0.0198 s for the baseline and 0.0204 s for the JIT run). It needs 11,345 iterations, against 7,033 for the JIT run, which starts each bond length from the previous solution. The JIT run therefore finishes first (143.80 s against 164.91 s). These two runs differ in device, compilation, and starting point at the same time, so the comparison does not isolate the effect of the GPU.

5.2.5 Implementation 4: CPU vs GPU Scaling Study (4–26 Qubits)

We ran a scaling study with a separate CPU stack and GPU stack at qubit counts from 4 to 26. Table 3 lists the results.

Qubits	State Vector	CPU (s)	GPU (s)	CPU/GPU ratio
4	256 B	8.33	0.79	10.5
8	4 KB	6.54	1.07	6.1
12	64 KB	4.24	1.20	3.5
14	256 KB	9.10	0.90	10.1
16	1 MB	5.67	0.85	6.7
18	4 MB	12.03	0.84	14.3
20	16 MB	46.77	1.08	43.2
22	64 MB	161.07	2.21	72.9
24	256 MB	478.73	5.87	81.5
26	1 GB	1425.06	17.71	80.5

Table 3: CPU and GPU runtimes in the scaling study. State vector size is $2^n \times 16$ bytes (complex128). The ratio compares two software stacks that differ in more than the device (see text).

Both paths use the same transverse-field Ising Hamiltonian with coupling $J = 1.0$ and field $h = 0.5$, and a hardware-efficient ansatz with two layers of R_Y and R_Z rotations followed by a CNOT chain. The GPU path has the lower runtime at every tested size, and the CPU/GPU time ratio ranges from 3.5 at 12 qubits to 81.5 at 24 qubits. The two paths differ in more than the device. The CPU path uses `lightning.qubit` through the JAX interface with `jax.jit` and the

Optax Adam optimizer, with JAX in its default 32-bit mode. The GPU path uses `lightning.gpu` through the autograd interface with adjoint differentiation and the PennyLane Adam optimizer. The ratio therefore compares two software stacks, not two devices. At 4 qubits the state vector holds 16 amplitudes, and we interpret the ratio of 10.5 at that size as overhead in the CPU stack rather than as a hardware effect.

Figure 2 shows the scaling comparison.

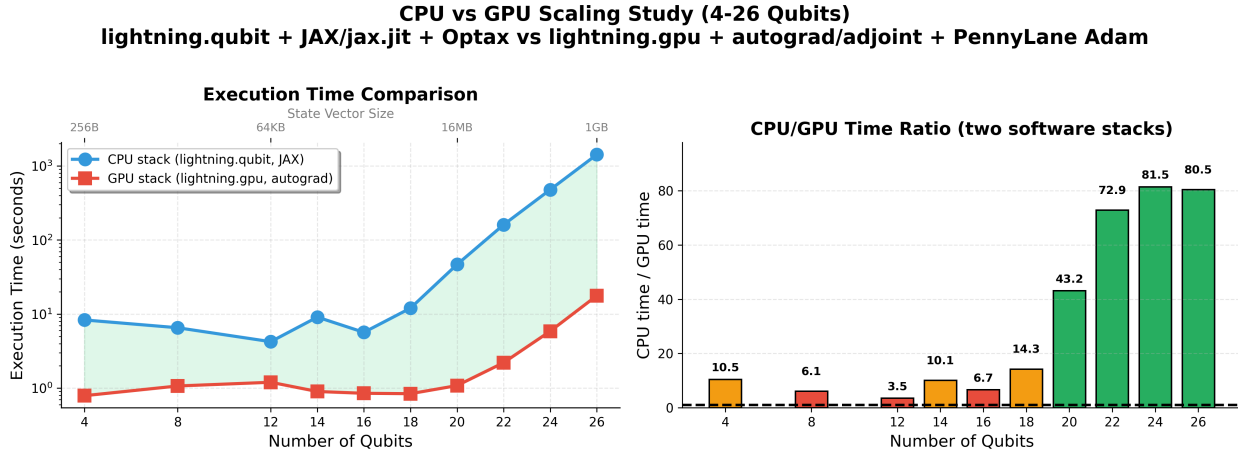


Figure 2: CPU and GPU stacks in the scaling study from 4 to 26 qubits. Left: runtime (log scale). Right: CPU/GPU time ratio.

5.2.6 Implementation 5: MPI Parallelization

The MPI implementation (`vqe_mpi.py`) splits the 100 bond lengths into contiguous blocks, one per rank, and each rank runs the Optax and Catalyst JIT runner of Implementation 2 on its block. Table 4 lists the measured runtimes.

Ranks	Runtime (s)
2	8.45
4	6.07
8	5.48
16	5.06
32	5.04

Table 4: MPI runtimes for the 100-bond-length scan. Each value is the elapsed time on rank 0, measured before the final gather.

We do not report a parallel speedup for these runs. The serial Optax and JIT job requested one core (`ppn=1`), and the MPI jobs ran under separate allocations. The per-bond times of the serial run assign 73.5 s of work to rank 0 and 70.3 s to rank 1 at 2 ranks, while the measured 2-rank runtime is 8.45 s. A balanced split of the same work cannot explain this factor of 8.7, so the serial and MPI jobs had different resources available, for example the number of cores each process could use. The timer also covers rank 0 only. We cannot rerun both jobs under one allocation, so the ratio of their runtimes mixes the effect of the rank count with the effect of the allocation. Among the MPI runs, the runtime falls from 8.45 s at 2 ranks to 5.04 s at 32 ranks.

5.2.7 Implementation 6: Multi-GPU Runs

The multi-GPU benchmark uses a separate test problem: a sum of Pauli- Z operators on n qubits, a layer of R_Y rotations (with an additional R_Z layer in the maximum-size test) followed by a CNOT chain, `lightning.gpu` with adjoint differentiation, and gradient descent with step size 0.1. One MPI rank drives each GPU.

Maximum size. On one H100, 5 iterations at 26, 28, and 29 qubits take 0.62 s, 2.46 s, and 5.07 s per iteration. The 30-qubit run fails with an out-of-memory error. Figure 3 shows these times together with an estimate of the GPU memory, which we compute as four times the $2^n \times 16$ -byte state vector. The estimate gives 64 GB at 30 qubits, below the 80 GB of the H100, so it undercounts the memory that `lightning.gpu` allocates.

Independent problems. In the scaling test, each of the 4 GPUs solves 2 independent 22-qubit problems with 15 iterations each. The longest per-GPU time is 8.04 s, and the sum of the four per-GPU times is 31.99 s. Their ratio, 3.98, shows that the four GPUs carried equal load. It is not a measured speedup, because we did not run the 8 problems on a single GPU. In the throughput test, each GPU solves 3 independent 20-qubit problems, and the longest per-GPU time is 12.27 s.

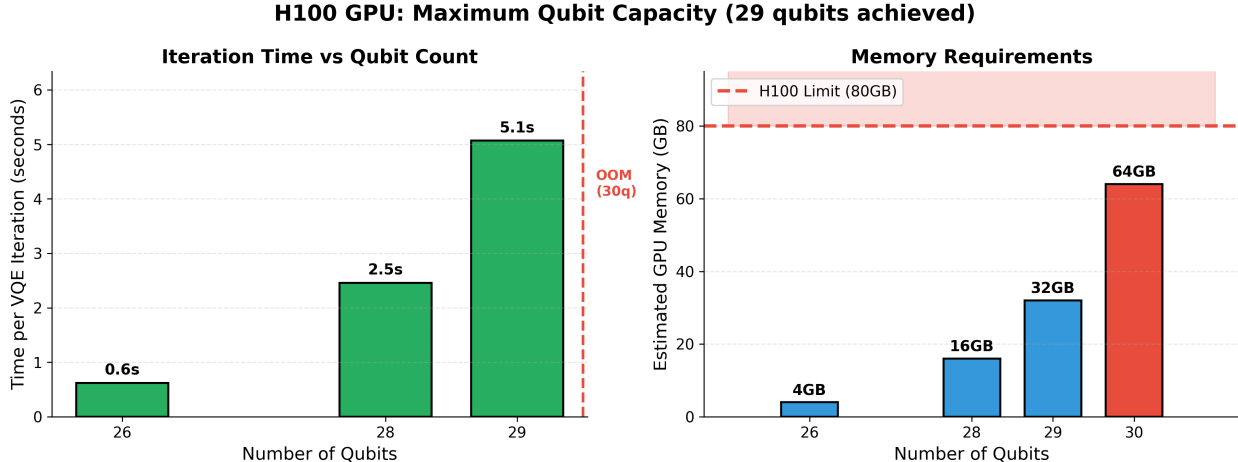


Figure 3: Maximum problem size on one NVIDIA H100. Left: measured time per iteration at 26, 28, and 29 qubits. Right: estimated GPU memory, four times the complex128 state vector. The 30-qubit run failed with an out-of-memory error.

6 Discussion

6.1 Physical Interpretation of Results

The computed potential energy surface captures the essential quantum chemistry of the H_2 molecule:

Equilibrium Geometry: The minimum energy occurs near $d_{eq} \approx 0.74 \text{ \AA}$, matching the experimental value of $0.741 \text{ \AA}$.

Bonding Energy: At equilibrium, the VQE energy is approximately -1.137 Hartree. The exact STO-3G value is -1.1372 Hartree, so our single-parameter ansatz achieves good accuracy.

Dissociation Behavior: Beyond $2.5 \text{ \AA}$, the energy approaches the separated-atom limit. The curve shows the expected behavior, though the single-parameter ansatz has known limitations in the dissociation region where electron correlation is strong.

Ansatz Effectiveness: The double excitation ansatz with a single parameter works well for H_2 near equilibrium. The dominant correlation effect in H_2 is both electrons moving from the bonding (σ) to antibonding (σ^*) orbital, which is what the double excitation operator captures.

HPC Focus: The physical results above validate our VQE implementation, but this paper focuses on HPC parallelization. The quantum chemistry here is a representative workload; the parallelization techniques apply to other variational quantum algorithms.

6.2 Computational Performance Analysis

Serial Baseline: The preliminary serial run (Table 2) achieves 157.98 circuit evaluations per second. On the cluster, the baseline takes 593.95 seconds for 100 bond lengths with 300 iterations each.

Bottleneck Identification: Profiling with Python’s `cProfile` confirmed that VQE optimization dominates runtime. Gradient computation (`_grad_with_forward`) and cost function evaluation (`qnode.__call__`) account for 28.2 seconds of the 32.7 second total (86%). Excluding I/O and plotting, over 95% of work is in the VQE loops ($f_p \approx 0.95$), which is favorable for parallelization.

Iteration Count: On one core, the largest runtime reduction comes from fewer iterations. Early stopping, a larger learning rate, and warm starts cut the iteration count by a factor of 4.3 and the runtime by a factor of 4.13 (Implementation 2).

GPU: At 4 qubits, `lightning.gpu` has the lowest time per iteration (0.0145 s), but it runs 11,345 iterations from cold starts and finishes after the JIT run. In the scaling study, the GPU stack is faster at every size from 4 to 26 qubits, with a CPU/GPU time ratio between 3.5 and 81.5, for two stacks that differ in interface, differentiation method, and optimizer.

MPI: Runs with 2 to 32 ranks take between 8.45 s and 5.04 s. We do not compare them with the serial runs, for the reasons in Implementation 5.

Multi-GPU: One H100 runs up to 29 qubits. Independent problems spread evenly across 4 GPUs (sum of per-GPU times 31.99 s, longest per-GPU time 8.04 s). A single-GPU run of the same problems is needed to measure a speedup.

6.3 Relevance to Original Questions

Question 1: VQE Accuracy

VQE with a single-parameter double excitation ansatz computes the H_2 potential energy surface in the STO-3G basis, with an energy of about -1.137 Ha at the equilibrium bond length (Figure 1).

Question 2: HPC Parallelization

The H_2 scan parallelizes across bond lengths with no communication between ranks, and the MPI runs complete the scan in 5.04 s to 8.45 s. Our measurements do not isolate the parallel speedup, because the serial and MPI jobs ran under different allocations (Implementation 5). On one core, the stopping rule, learning rate, and starting point change the runtime by a factor of 4.13 while the time per iteration stays between 0.0198 s and 0.0204 s. For 20 to 26 qubits, the GPU stack in the scaling study runs 43.2 to 81.5 times faster than the CPU stack.

7 Conclusions

We implemented VQE for the hydrogen molecule potential energy surface and measured four implementations on the ERAU Vega cluster with 4 NVIDIA H100 GPUs:

1. **Algorithm Validation:** The single-parameter double excitation ansatz reproduces the H_2 potential energy surface, with ~ -1.137 Ha at $0.74 \text{ \AA}$.

2. **Iteration Count:** Early stopping, a larger learning rate, and warm starts reduce the iterations from 30,000 to 7,033 and the runtime from 593.95 s to 143.80 s. The time per iteration stays between 0.0198 s and 0.0204 s, so the reduction tracks the iteration count.
3. **GPU:** At 4 qubits, `lightning.gpu` lowers the time per iteration to 0.0145 s. In the 4 to 26 qubit study, the CPU/GPU time ratio ranges from 3.5 to 81.5, for two software stacks that differ in interface, differentiation method, and optimizer.
4. **MPI:** 2 to 32 ranks complete the scan in 8.45 s to 5.04 s. The serial and MPI jobs ran under different resource allocations, so we report no MPI speedup.
5. **Multi-GPU:** One H100 runs up to 29 qubits, and 30 qubits fails with an out-of-memory error. Eight independent 22-qubit problems on 4 GPUs take at most 8.04 s per GPU, against a sum of 31.99 s over the four GPUs. We did not run them on one GPU.
6. **Practices:** Report the optimizer, stopping rule, and starting point with each runtime, because these settings changed the runtime by a factor of 4.13 in our runs. Run the serial baseline and the parallel runs under one resource allocation, and time the slowest rank. Measure a single-device run before reporting multi-device efficiency.

7.1 Broader Impact

The bond-length scan is a parameter sweep in which each point is an independent optimization. The same distribution across ranks or GPUs applies to other sweeps of this form, for example scans over several geometric parameters.

8 Changes in Version 2

Version 1 of this preprint attributed a $4.13\times$ runtime reduction to JIT compilation, reported an MPI speedup of $28.5\times$ and a combined speedup of $117\times$, and reported 99.4% multi-GPU parallel efficiency. Version 2 corrects these statements. The $4.13\times$ reduction tracks a $4.3\times$ reduction in iteration count from early stopping, a larger learning rate, and warm starts. The MPI speedups compared jobs with different resource allocations, and version 2 withdraws them; the measured MPI runtimes remain. The 99.4% figure divided the sum of per-GPU times by four times the longest per-GPU time, so it measured load balance, not speedup. Version 2 also corrects the description of the scaling-study Hamiltonian (transverse-field Ising, not a sum of Pauli-Z operators), states the software differences between the CPU and GPU paths, and removes the figure built on the withdrawn speedups.

9 Acknowledgements

We thank Dr. Khanal for guidance on parallelization strategies and HPC methodologies. This work was conducted on the ERAU Vega HPC cluster featuring AMD EPYC 9654 processors and NVIDIA GPU accelerators. The quantum simulations used the PennyLane quantum computing framework with Lightning backend, JAX for automatic differentiation, and Catalyst for JIT compilation.

Large language model tools were used to assist with manuscript preparation and editing. The authors take full responsibility for all scientific content, methodology, and results.

10 Codebase

All source code can be accessed at <https://github.com/rylanmalarchick/QuantumVQE>

References

- [1] A. Peruzzo, et al., *A variational eigenvalue solver on a photonic quantum processor*, Nature Communications **5**, 4213 (2014).
- [2] J. Romero, R. Babbush, J. R. McClean, C. Hempel, P. J. Love, and A. Aspuru-Guzik, *Strategies for quantum computing molecular energies using the unitary coupled cluster ansatz*, Quantum Science and Technology **4**, 014008 (2018).
- [3] A. Kandala, et al., *Hardware-efficient variational quantum eigensolver for small molecules and quantum magnets*, Nature **549**, 242–246 (2017).
- [4] V. Bergholm, et al., *PennyLane: Automatic differentiation of hybrid quantum-classical computations*, arXiv:1811.04968 (2018).
- [5] Xanadu, *PennyLane-Lightning: Fast state-vector simulator*, <https://github.com/PennyLaneAI/pennylane-lightning> (2024).
- [6] NVIDIA Corporation, *cuQuantum SDK: High-performance libraries for quantum computing simulation*, <https://developer.nvidia.com/cuquantum-sdk> (2023).
- [7] J. Bradbury, et al., *JAX: Composable transformations of Python+NumPy programs*, <https://github.com/google/jax> (2018).
- [8] M. Valiev, et al., *NWChem: A comprehensive and scalable open-source solution for large scale molecular simulations*, Computer Physics Communications **181**, 1477–1489 (2010).
- [9] V. Bergholm, et al., *PennyLane: Automatic differentiation of hybrid quantum-classical computations*, arXiv:1811.04968 (2018).
- [10] S. McArdle, S. Endo, A. Aspuru-Guzik, S. C. Benjamin, and X. Yuan, *Quantum computational chemistry*, Reviews of Modern Physics **92**, 015003 (2020).
- [11] M. Cerezo, et al., *Variational quantum algorithms*, Nature Reviews Physics **3**, 625–644 (2021).
- [12] A. Szabo and N. S. Ostlund, *Modern Quantum Chemistry: Introduction to Advanced Electronic Structure Theory*, Dover Publications (1996).
- [13] G. M. Amdahl, *Validity of the single processor approach to achieving large scale computing capabilities*, AFIPS Conference Proceedings **30**, 483–485 (1967).